

Metodologie di Programmazione

Come funziona Java (JDK, JVM, JRE)

Java funziona utilizzando diversi componenti che consentono la compilazione del codice (tramite il compilatore javac contenuto della JDK), la traduzione del codice sorgente in codice binario (bytecode), e l'esecuzione effettiva (all'interno della JVM).

JDK (Java Development Kit)

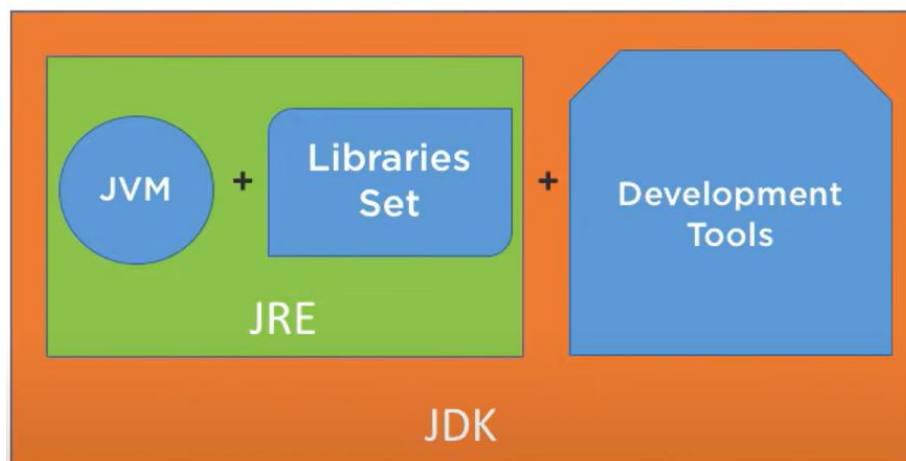
La JDK è l'ambiente di sviluppo di Java. **Esso fornisce gli strumenti necessari per sviluppare applicazioni Java**, tra cui il compilatore, chiamato **javac**, che riceve il file in formato **.java** contenente il codice sorgente e lo traduce in codice binario, o bytecode, restituendo un file con estensione **.class** eseguibile dalla JVM.

JVM (Java Virtual Machine)

La JVM è l'ambiente di esecuzione di Java. **Essa interpreta e esegue il bytecode Java**. Nella pratica: riceve il file **.class**, contenente il codice compilato (bytecode) generato dal compilatore javac, e lo esegue all'interno della Macchina Virtuale (per l'appunto JVM). Questo concetto di macchine virtuale **permette la Portabilità del Codice**, consentendo di eseguire i programmi Java su qualunque macchina senza la necessità di apportare modifiche (*principio "Write Once, Run Anywhere"*).

JRE (Java Runtime Environment)

La JRE è l'ambiente di runtime di Java. **Essa include la JVM e le librerie Java necessarie per eseguire il nostro codice**. Non include strumenti di sviluppo, come il compilatore Java, ma fornisce tutto il necessario per eseguire le applicazioni.



Tipi Primitivi

I **Tipi Primitivi** (o *tipi di dati di base*) sono built-in, ovvero sono predefiniti nel linguaggio di programmazione. Nel nostro caso, nel linguaggio programmazione Java, i tipi primitivi **sono 8**:

Tipo	Intervallo	Dimensione
double	parte intera: ± 10308 , parte frazionaria: circa 15 cifre decimali significative	8 byte
long	-9223372036854775808...9223372036854775807	8 byte
int	-2147483648...2147483647	4 byte
float	parte intera: ± 1038 , parte frazionaria: circa 7 cifre decimali significative	4 byte
boolean	true, false	1 byte
char	Rappresenta tutti i caratteri codificati con Unicode	2 byte
short	-32768...32767	2 byte
byte	-128...127	1 byte

Fondamentale specificare che **le Stringhe** (String), sebbene siano un insieme di caratteri (char - tipo primitivo), **NON sono un tipo primitivo in Java!!!**

Letterali

Un letterale è una rappresentazione a livello di codice sorgente del valore di un tipo di dati:

- **27** è un letterale per gli **interi**
- **3.14** è un letterale per i **double**
- **true** o **false** sono gli unici due letterali per il tipo **booleano**
- **“Ciao, mondo”** è un letterale per il tipo **String**

Variabili ed Identificatori

Il nome assegnato a una variabile è detto **identificatore**, ovvero una sequenza di lettere, cifre, `_` e `$`. Gli identificatori sono **case-sensitive** (*VaRiaBile e variabile sono considerati due identificatori distinti*). Non possono essere utilizzate alcune parole riservate (es. public, static, int, double, ecc.) e si utilizza la notazione a cammello (Camel case).

Ogni variabile viene creata mediante **dichiarazione** ed assume un valore tramite **assegnazione**.

```
int contatore = 0;
```

Se volessimo analizzare cosa avviene a “livello temporale”:

	a	b	c
int a, b;	non definita	non definita	
a = 5;	5	non definita	
b = a+10;	5	15	
int c = a+b;	5	15	20
a = c-3;	17	15	20

Conversioni tra Tipi

In ogni linguaggio di programmazione è fondamentale conoscere ed apprendere a pieno come avviene la conversione tra tipi, nota anche come **casting**, per poter lavorare e gestire tipi di dati diversi. Java in particolare supporta due principali tipi di conversione:

Conversioni Implicite: Le conversioni implicite sono gestite dal compilatore e avvengono quando il tipo di dati a cui si vuole arrivare “è più grande” di quello originario, e quindi il cast può avvenire senza perdita di informazione.

(Es. conversione da int a double, oppure da short a long, da char a int etc...)

La conversione implicita avviene anche quando si vuole passare un oggetto di una sottoclasse a una superclasse, cioè “da sotto a sopra”. Questo è possibile perché ogni oggetto di una sottoclasse è anche un oggetto della sua superclasse. Tuttavia questo comporta che, con il cast, si andranno a perdere tutti i metodi aggiuntivi della sotto-classe. Esempio:

```
class Animale {
    void faiVerso() {
        System.out.println("Verso generico");
    }
}

class Cane extends Animale {
    void abbaia() {
        System.out.println("Abbaia");
    }
}

public class Main {
    public static void main(String[] args) {
        Cane cane = new Cane();
        Animale animale = cane; // Conversione implicita
        animale.faiVerso(); // Funziona, ma non puoi chiamare abbaia() su animale
    }
}
```

Conversioni Esplicite: Le conversioni esplicite sono necessarie quando si desidera passare da un tipo “grande” ad uno “più piccolo” comportando una perdita di dati. In questi casi, è necessario specificare il tipo di destinazione tra parentesi.

(Es. se voglio passare una variabile da double a int dovrò scrivere -> *int variabileInt = (int) variabileDouble*)

Anche per quanto riguarda i passaggi da superclasse a sottoclasse è necessario un Cast Esplicito, per consentire al nostro oggetto di “apprendere” i nuovi metodi. Esempio:

```
class Animale {
    void faiVerso() {
        System.out.println("Verso generico");
    }
}

class Cane extends Animale {
    void abbaia() {
        System.out.println("Abbaia");
    }
}

public class Main {
    public static void main(String[] args) {
        Animale animale = new Cane(); // Conversione implicita
        Cane cane = (Cane) animale; // Cast esplicito
        cane.abbaia(); // Funziona, ora puoi chiamare abbaia()
    }
}
```

“Gerarchia” dei tipi

Conversione
Esplicita

1. ``double``: 64-bit floating-point number
2. ``float``: 32-bit floating-point number
3. ``long``: 64-bit integer
4. ``int``: 32-bit integer
5. ``short``: 16-bit integer
6. ``byte``: 8-bit integer
7. ``char``: 16-bit Unicode character

Conversione
Implicita

Classi Wrapper

IMPORTANTISSIMO: In Java, i tipi primitivi (come `int`, `double`, `char`, ecc.) non sono oggetti. Per gestire i tipi primitivi come oggetti e per fornire metodi utili, Java mette a disposizione delle classi chiamate "classi wrapper". Queste classi wrapper sono fondamentali per utilizzare i tipi primitivi all'interno delle collezioni di Java, come `ArrayList`, `HashMap`, e `HashSet`, che possono contenere solo oggetti e non tipi primitivi.

Ogni tipo primitivo ha la sua Classe Wrapper corrispondente: `int` → `Integer`, `double` → `Double`, `char` → `Character`, `boolean` → `Boolean` etc...

Le classi wrapper offrono anche metodi statici utili per la conversione tra tipi primitivi e stringhe. Alcuni esempi includono:

`Integer.parseInt(String str)` che converte una stringa (es. `"7"`) nel relativo numero intero (7).

`Boolean.parseBoolean(String str)` che converte una stringa (es. `"true"`) nel relativo booleano (`true`).

E tanti altri `Qualcosa.parseQualcosa(String str)` per convertire da stringa a Qualcosa.

L'**autoboxing** converte automaticamente un tipo primitivo al suo tipo wrapper associato.

L'**auto-unboxing** converte automaticamente da un tipo wrapper all'equivalente tipo primitivo

Conversioni da Dati Primitivi a Stringhe

Allo stesso modo di quanto visto fino ad ora anche la classe `String` è una Classe Wrapper e possiede alcuni metodi statici utili come quelli per passare da tipo primitivo a stringa:

`String.valueOf(int i)` che converte l'intero `i` (es. 12) in formato stringa (`"12"`)

`String.valueOf(boolean b)` che converte bla bla bla...

Oppure si possono chiamare i metodi **`toString()`** delle varie altre classi Wrapper (es. `Boolean.valueOf(true).toString()`)

Oppure tramite **concatenazione di stringhe** (es. `"Pokemon di lv: " + 7`) che converte in automatico ogni valore in stringa. Piccolo Trick: si può tranquillamente fare concatenazione con stringa vuota (tipo `7 + ""` restituisce `"7"`).

Incapsulamento

L'incapsulamento è il processo di nascondere i dettagli implementativi interni di un oggetto e di esporre solo l'interfaccia pubblica, cioè l'insieme di metodi e funzionalità necessarie agli utenti per interagire con quell'oggetto. In Java, l'incapsulamento è implementato principalmente attraverso i **modificatori di accesso** ed i metodi **getter e setter**.

Modificatori di Accesso:

- **public**: accessibile a tutti
- **private**: accessibile solo all'interno della classe
- **protected**: accessibile all'interno della classe e alle sue sottoclassi
- **default**: (se non viene impostato altro modificatore) accessibile solo all'interno dello stesso package

In sintesi, l'incapsulamento in Java aiuta a garantire che il **codice sia più sicuro, mantenibile e facile da comprendere**, promuovendo una progettazione più modulare e controllata. Più nello specifico garantisce:

1. **Controllo sull'accesso dei dati**: L'incapsulamento permette di controllare come i dati vengono letti e modificati. I dati privati non possono essere cambiati direttamente da fuori la classe, riducendo il rischio di modifiche non desiderate.
2. **Manutenzione e Modifica**: Modificare la struttura interna di una classe non influisce su altre classi che usano quella classe, se i cambiamenti sono fatti attraverso i metodi pubblici. Questo facilita la manutenzione e l'evoluzione del codice.
3. **Protezione dell'Integrità dei Dati**: Le variabili private possono essere protette e validate tramite metodi setter, garantendo che solo valori validi vengano assegnati. Per esempio se ho una classe orologio nel mio metodo setter posso andare a mettere dei controlli affinché non possa essere assegnata un'ora tipo 27:92

Anatomia della Memoria Java

Heap

L'**Heap** è l'area della memoria utilizzata per la gestione dinamica degli oggetti e delle istanze di classi. È condiviso tra tutti i thread e ha una dimensione molto più grande rispetto allo stack. Le caratteristiche principali sono 3:

1. **Memoria Dinamica**: Gli oggetti creati con l'operatore new vengono allocati nell'heap.
2. **Garbage Collection**: La memoria nell'heap è gestita tramite garbage collection, che libera la memoria non più utilizzata per evitare perdite di memoria. La JVM identifica e rimuove gli oggetti che non sono più referenziati.
3. **Dimensione Variabile**: La dimensione dell'heap può essere configurata tramite parametri della JVM e può essere estesa per gestire grandi quantità di dati.

Stack

Lo **Stack** è una struttura dati a LIFO (Last In, First Out) utilizzata per la gestione della memoria temporanea. Principalmente lo stack contiene i **frame di attivazione dei metodi**, che includono i parametri, le variabili locali, il valore/indirizzo di ritorno ed i riferimenti agli oggetti allocati nello heap.

Esempio: Quando un metodo viene chiamato, viene creato un nuovo frame di attivazione nello stack. Le variabili locali e i parametri del metodo vengono memorizzati temporaneamente in questo frame fino al termine del metodo, quando il frame viene rimosso dallo stack.

MetaSpace

Il **MetaSpace** è un'area di memoria separata dall'Heap, utilizzata per memorizzare le **metainformazioni**, cioè tutte le informazioni necessarie alla JVM per gestire le classi. Nella pratica contiene informazioni riguardanti la struttura delle classi, i campi statici e i metodi (sia statici che non).

Il MetaSpace, al contrario del suo predecessore PermGen, può aumentare dinamicamente le sue dimensioni basandosi sulla memoria fisica disponibile, riducendo così la probabilità di esaurimento.

Perché il metodo main() è dichiarato public static void?

- **public:** Permette alla JVM di accedere al metodo.
- **static:** La Java Virtual Machine invoca il metodo main ancora prima di aver creato qualsiasi oggetto. Dunque è fondamentale che il metodo sia static senza essere legato ad alcun'istanza.
- **void:** Il metodo non restituisce un valore, sufficiente per il suo scopo di avvio del programma.

Interfacce

Oltre alla sottoclassificazione, Java supporta un altro meccanismo di ereditarietà chiamato **sottotipizzazione**.

La sottotipizzazione permette di specificare una relazione tra classi altrimenti non correlate, definendo un'interfaccia: cioè un insieme di metodi comuni che ogni classe implementante deve contenere.

Nelle interfacce, si definiscono solo i nomi dei metodi e i loro valori di ritorno, **SENZA** specificarne il corpo. Questi metodi, privi di implementazione, sono chiamati **metodi astratti**. Tuttavia, a partire da Java 8, tramite la parola chiave **default**, è possibile assegnare anche un corpo ai metodi delle interfacce. Ciò consente alle classi che implementano l'interfaccia di ereditare a tutti gli effetti una versione di default del metodo, pur rimanendo libere di sovrascriverlo se necessario. Inoltre, è possibile dichiarare **metodi statici** nelle interfacce, che possono essere richiamati direttamente senza bisogno di un'istanza della classe implementante.

Questo rende le interfacce simili alle classi astratte, con alcune differenze chiave. Ad esempio, né le interfacce né le classi astratte possono essere istanziate. Tuttavia, le classi astratte possono dichiarare campi e fornire costruttori, che saranno ereditati dalle sottoclassi. **Le interfacce, invece, non possono contenere campi di istanza o costruttori.**

Un'altra differenza fondamentale sta nel fatto che una classe può estendere 1 sola superclasse, MA può implementare più interfacce consentendo così una sorta di "ereditarietà multipla". Questo è particolarmente utile per combinare comportamenti e funzionalità in classi che idealmente non hanno molto in comune e difficilmente si troveranno nella stessa gerarchia delle classi. Per esempio: una macchina ed un essere umano, hanno poco in comune eppure entrambe possono implementare la stessa interfaccia `Corsa()`. Di fatto entrambe possono correre, ma in maniere completamente diverse. Proprio per questo motivo i metodi delle interfacce sono astratti e richiedono l'implementazione concreta all'interno delle singole classi.

Ultima importante annotazione è che **le interfacce possono estendere altre interfacce**, consentendo di costruire gerarchie complesse di interfacce. Ad esempio, un'interfaccia `Animale` potrebbe estendere un'interfaccia `EssereVivente`, ereditando così i suoi metodi.

Interfacce Funzionali

In Java 8 è disponibile una nuova annotazione **@FunctionalInterface**. Quest'annotazione garantisce che l'interfaccia sia dotata **esattamente di un solo metodo astratto**, sebbene possa comunque contenere altri metodi default o statici. Di fatto le Interfacce Funzionali vengono dette di tipo SAM (Single Abstract Method)

```
@FunctionalInterface
interface Operazione {

    int esegui(int a, int b);

    default void stampaRisultato(int risultato) {
        System.out.println("Il risultato è: " + risultato);
    }

    static void descrivi() {
        System.out.println("Questa è un'interfaccia funzionale per operazioni matematiche.");
    }
}
```

In questo caso, `Operazione` è un'interfaccia funzionale perché contiene un solo metodo astratto `esegui`, che prende due parametri interi. Tuttavia non abbiamo idea di come queste due variabili vadano utilizzate. Per questo motivo la nostra Interfaccia può essere utilizzata in un contesto in cui è necessario **passare una funzione come argomento**, e per farlo sarà necessario utilizzare le **espressioni lambda**.

Espressioni Lambda

Le espressioni lambda permettono di fornire implementazioni di interfacce funzionali in modo conciso e rapido. E' da consigliare **l'impiego delle espressioni lambda principalmente quando il codice si scrive su una sola riga**. In alternativa, si preferisce un'implementazione mediante classe o classe anonima.

```
public static void main(String[] args) {  
    // Uso di un'espressione lambda per implementare l'interfaccia funzionale  
    Operazione somma = (a, b) -> a + b;  
  
    int risultato = somma.esegui(1, 4);  
    somma.stampaRisultato(risultato);  
}
```

La sintassi per creare le espressioni lambda è:

(parametri della funzione) -> {corpo della funzione}

Il tipo dei parametri in input è opzionale. Le parentesi tonde sono opzionali se in input abbiamo un solo parametro. Le parentesi graffe intorno al codice della funzione sono opzionali se è costituito da una sola riga.

```
() -> { System.out.println("hello, lambda!"); };  
(int a, int b) -> { return a+b; }  
(a, b) -> { return a+b; }  
(a, b) -> return a+b;  
(a, b) -> a+b;
```

Riferimenti a Metodi (method references)

I **riferimenti ai metodi** in Java permettono di riferirsi direttamente a metodi o costruttori esistenti senza doverli invocare. Questi riferimenti possono essere utilizzati come espressioni lambda in modo più conciso e leggibile, specialmente quando una lambda esegue un'operazione già definita da un metodo esistente. Sintassi:

ClassName::staticMethodName

oppure

instance::instanceMethodName

I riferimenti ai metodi sono essenzialmente scorciatoie per esprimere espressioni lambda che chiamano un metodo esistente.

Per esempio:

- La lambda `x -> Integer.valueOf(x)` può essere riscritta come `Integer::valueOf`.
- La lambda `() -> new ArrayList<>()` può essere riscritta come `ArrayList::new`.

Interfacce Funzionali built-in

Predicate<T>

Rappresenta una funzione che prende un argomento di tipo T e restituisce un valore booleano (true o false). Viene utilizzato per esprimere condizioni o filtri.

```
Predicate<String> isEmpty = String::isEmpty;  
boolean result = isEmpty.test(""); // Restituisce true perché la stringa è vuota
```

Function<T, R>

Rappresenta una funzione che prende un argomento di tipo T e restituisce un valore di tipo R. È utilizzata per trasformare un dato da un tipo ad un altro.

```
Function<String, Integer> stringLength = String::length;  
int length = stringLength.apply("Hello"); // Restituisce 5
```

Consumer<T>

Rappresenta una funzione che accetta un argomento di tipo T ma non restituisce alcun risultato. Viene usato per eseguire operazioni su un dato, come stamparlo, modificarlo, ecc.

```
Consumer<String> print = System.out::println;  
print.accept("Hello World!"); // Stampa "Hello World!" sulla console
```

Supplier<T>

Rappresenta una funzione che non accetta argomenti ma restituisce un valore di tipo T. È utile quando devi generare o fornire un valore senza bisogno di input.

```
Supplier<Double> randomValue = Math::random;  
double value = randomValue.get(); // Restituisce un numero casuale tra 0.0 e 1.0
```

Queste interfacce sono molto potenti quando utilizzate insieme alle espressioni lambda e ai method references, rendendo il codice più conciso e leggibile, specialmente in operazioni di elaborazione dei dati come quelle eseguite sugli Stream.

Stream

Uno stream rappresenta una sequenza di elementi su cui possono essere effettuate una o più operazioni.

Gli stream non memorizzano i dati ma lavorano su collezioni di dati, come per esempio le collections. Le caratteristiche principali sono:

- **Non-Storage:** Gli stream non memorizzano i dati. Operano su dati forniti da una sorgente e producono risultati tramite operazioni senza modificare la sorgente.
- **Operazioni Lazy:** Le operazioni sugli stream sono lazy (pigre) perchè non vengono eseguite finché non viene invocata un'operazione terminale (come collect o forEach).

Inoltre gli stream supportano due diversi tipi di operazioni:

- **Operazioni Intermedie:** Modificano uno stream e restituiscono un nuovo stream. Esempi includono filter, map, distinct, sorted, e limit.
- **Operazioni Terminali:** Producono un risultato o un effetto collaterale. Esempi includono collect, forEach, reduce, count, e min/max.

Esempi:

- 1) Data una lista di nomi vogliamo prendere solo quelli che iniziano per "A". Stampa il risultato a video senza salvare nulla.

```
public static void main(String[] args) {
    List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David", "Eve");

    // Creare uno stream dalla lista
    names.stream()
        .filter(name -> name.startsWith("A"))
        .forEach(System.out::println); // Output: Alice
}
```

- 2) Data una lista di nomi vogliamo prendere solo quelli con lunghezza maggiore di 3, metterli tutti in maiuscolo ed ordinarli. Il risultato deve essere "collezionato" in una List e POI stampato a video.

```
public static void main(String[] args) {
    List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David", "Eve");

    List<String> filteredNames = names.stream()
        .filter(name -> name.length() > 3) // Operazione intermedia
        .map(String::toUpperCase)         // Operazione intermedia
        .sorted()                          // Operazione intermedia
        .collect(Collectors.toList());     // Operazione terminale

    System.out.println(filteredNames); // Output: [ALICE, CHARLIE, DAVID]
}
```

Operazioni Comuni sugli Stream

- **filter**(Predicate<T> predicate): Filtra gli elementi dello stream in base al predicato fornito.
- **map**(Function<T, R> mapper): Trasforma ogni elemento dello stream applicando la funzione di mapping.
- **reduce**(BinaryOperator<T> accumulator): Riduce gli elementi dello stream a un singolo valore. Per esempio `.reduce((a,b) -> a + b)` può restituire la somma (o la concatenazione) di tutti gli elementi contenuti nello stream.
- **distinct**(): Rimuove i duplicati dallo stream.
- **sorted**(): Ordina gli elementi dello stream in base all'ordinamento naturale o a un comparatore fornito.
- **limit**(long maxSize): Limita il numero di elementi dello stream.
- **count**(): Conta, e restituisce, il numero di elementi nello stream.
- **forEach**(Consumer<T> action): Esegue un'azione su ciascun elemento dello stream. Questa operazione terminale applica il consumer fornito a ogni elemento dello stream. Non restituisce nulla e viene utilizzata per effetti collaterali, come la stampa o l'accumulo di risultati.
- **collect**(Collector<T, A, R> collector): Raccoglie gli elementi dello stream in una struttura dati, come una lista o un set, mutabili

Classi Anonime

In Java è possibile definire anche classi anonime (ovvero senza nome) che implementano un'interfaccia o estendono una classe. Esse vengono tornano utili in determinati contesti (ad esempio, per creare un iteratore al volo) o per **creare un'istanza che usiamo una sola volta nel codice**, senza il bisogno di creare un apposito altro file .java.

La sintassi è: `TipoDaEstendere unicoRiferimentoAOggetto = new TipoDaEstendere() { //corpo della classe anonima }`

Esempio:

```
public interface Formula {  
  
    double calculate(int a);  
    default double sqrt(int a) { return Math.sqrt(a); }  
}
```

```
Formula formula = new Formula() {  
    //corpo della classe anonima. In questo caso  
    //implemento i metodi dell'interfaccia  
    @Override  
    public double calculate(int a) {return sqrt(a * 100);}  
};
```

Classi Annidate e Classi Interne

Le classi usate finora vengono dette top-level, dal momento che non sono contenute in altre classi. Infatti, come abbiamo già visto, ciascuna di queste classi richiede un file .java dedicato con lo stesso nome della classe.

Tuttavia **Java consente anche di scrivere classi all'interno di altre classi**, che si dividono in:

- **Classi Annidate** (o Nested Class) se sono dichiarate come classi static.
- **Classi Interne** (o Inner Class) se sono NON sono chiarate static.

Prima di poter creare un oggetto della classe interna è necessario istanziare la classe esterna (top-level) che la contiene. Come tutti i membri di una classe, le classi interne possono essere dichiarate public, protected o private

Ciascuna classe interna, infatti, ha un riferimento implicito all'oggetto della classe esterna che la contiene. Dalla classe interna è possibile accedere a tutte le variabili e a tutti i metodi della classe esterna.

Per disambiguare casi di ambiguità (es. campi con lo stesso nome nella classe interna ed esterna):

- Se dalla classe interna viene usato soltanto this si fa riferimento ai campi e ai metodi di quella classe.
- Per far riferimento alla classe esterna è necessario specificare *ClasseEsterna.this.nomeCamp*

Allo stesso modo, per istanziare la classe interna a partire da un'altra classe si utilizza la sintassi `riferimentoOggettoClasseEsterna.new ClasseInterna()`

Perché sono Utili?

Raggruppamento logico delle classi: Se una classe è utile solo ad un'altra classe, è logico inserirla al suo interno e tenere le due classi logicamente vicine

Incrementa l'incapsulamento: Una classe B annidata in A può accedere ai membri di A (anchese privati), ma B può essere nascosta al mondo esterno.

Codice più leggibile e più facile da mantenere.

Strutture Dati

Arrays

Un array rappresenta un gruppo di variabili (chiamate elementi) tutte dello stesso tipo.

- Dichiarazione: `int[] a;`

Nome dell'array

- Creazione senza valori:

`a = new int[10];`

Ogni elemento viene
inizializzato con il valore di
default (0, false, null)

Come in molti altri linguaggi di programmazione si accede a un elemento dell'array specificando il nome dell'array seguito dalla posizione (indice) dell'elemento tra parentesi quadre (es. `a[0]`).

Gli Array in Java sono **mutabili**, cioè è possibile modificare gli elementi al suo interno contenuti tramite semplice assegnazione (es. `a[2] = 17`) ma hanno **lunghezza fissa**, cioè non può essere modificata una volta che l'array è creato. Tuttavia è possibile creare un nuovo array con nuove dimensioni a partire da un array pre-esistente con il metodo statico:

`Arrays.copyOf(vecchioArray, int nuovaDimensione)`

Matrici (Array Bidimensionali)

È possibile specificare un array a 2 dimensioni (o matrice) semplicemente specificando due coppie di parentesi quadre, in cui l'accesso avviene semplicemente specificando i due indici (prima righe e poi colonne) a cui si vuole accedere.

`String[][] matrice = new String[RIGHE][COLONNE];`

Iterable<T> e Iterator<T>

Prima di passare a vedere le collections di Java è fondamentale comprendere a pieno le interfacce `Iterable` e `Iterator`, dal momento che sono presenti in tutte quante le collezioni. In particolare ogni classe che implementa `Iterable` (cioè ogni classe "Iterabile") dispone del metodo `forEach` e devono necessariamente fornire un'implementazione al metodo astratto `iterator()` che serve a fornire un'istanza di `Iterator<T>` in grado di scorrere gli elementi della nostra classe.

```
public interface Iterable<T> {  
    Iterator<T> iterator();  
}
```

Per semplicità ho ommesso il corpo del metodo default `forEach`

Tuttavia, essendo `Iterator<T>` un'interfaccia, ha molto senso chiedersi:

"Come faccio a ritornare un oggetto di tipo `Iterator<T>` se l'interfaccia non può essere istanziata?"

Quello che si fa infatti è ritornare **un'istanza di una "sottoclasse concreta" che implementa `Iterator<T>`**. Per esempio, nel caso degli `ArrayList`, il metodo `iterator()` restituisce un oggetto di tipo `ArrayListIterator`. Stessa cosa per gli `HashSet` che ritornano un `HashSetIterator`; per i `TreeSet` che ritornano un `TreeSetIterator`; e così via...

"Se volessi creare la mia specifica classe che implementa `Iterable`?"

Dovrò fornire un'implementazione al metodo `iterator()` che ritorna l'istanza di un'altra classe (che dovremo creare) che implementa `Iterator<T>`.

```
public interface Iterator<T> {  
    boolean hasNext();  
    T next();  
    void remove();  
}
```

Esempio: Implementare il pattern Iterable in una classe generica Pila<T>

Step 1: Implementare Iterable<T> nella classe principale e fornire un'implementazione al metodo iterator():

```
public class Pila<T> implements Iterable<T>{

    private List<T> stack;

    public Pila() {
        stack = new ArrayList<T>();
    }

    @Override
    public Iterator<T> iterator() {
        return new PilaIterator();
    }
}
```

Step 2: creare la nostra personale Classeliterator implements Iterator<T> e fornire l'implementazione dei metodi hasNext() e next(). Solitamente si consiglia di crearla come classe interna privata così da migliorare l'incapsulamento, la compattezza del codice ed avere accesso a tutti i campi della classe principale.

```
private class PilaIterator implements Iterator<T>{

    private int i;

    public PilaIterator() {
        i = stack.size() - 1;
    }

    @Override
    public boolean hasNext() {
        return i >= 0;
    }

    @Override
    public T next() {
        if (hasNext()) {return stack.get(i--);}
        throw new NoSuchElementException();
    }
}
```

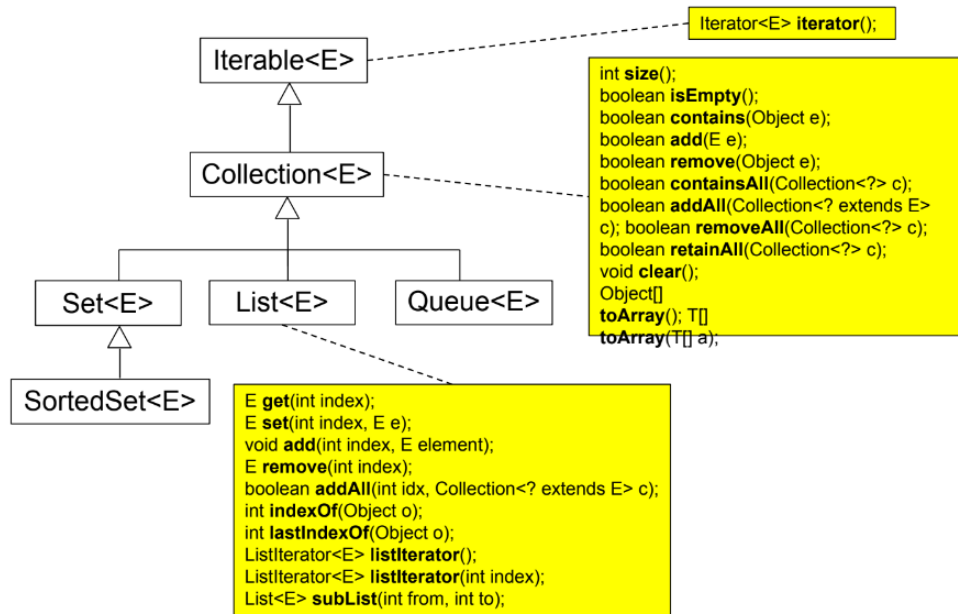
Step 3: utilizzare l'iterator:

```
public static void main(String[] args) {
    Pila<Integer> pila = new Pila<>();
    pila.push(11);
    pila.push(33);
    pila.push(22);

    Iterator<Integer> iterator = pila.iterator();
    while (iterator.hasNext()) {
        System.out.println(iterator.next());
    }
}
```

ATTENZIONE! Il metodo iterator() della classe principale restituisce un Iterator<T>. Dunque non si può (e non si deve) fare riferimento alla nostra personale Classeliterator interna e privata.

Java Collections



Le **Java Collections** sono un insieme di interfacce, classi e algoritmi che permettono di gestire gruppi di oggetti in modo efficiente e flessibile. Per **iterare** le Collezioni esistono diversi modi:

1. Mediante gli **Iterator**:

```
// "vecchio" stile, ma maggiore controllo
Iterator<Integer> i = collezione.iterator();
while(i.hasNext())
{
    int k = i.next();
    System.out.println(k);
}
```

Finché c'è ancora un elemento

Ottieni il prossimo elemento

2. Mediante il costrutto **"for each"**:

```
// "nuovo" stile: for each, più elegante
for (Integer k : collezione)
    System.out.println(k);
```

Per ogni elemento k nella collezione

3. Mediante **indici** (solo su liste):

```
// su collezioni che implementano List: elemento per elemento
for (int j = 0; j < collezione.size(); j++)
{
    int k = collezione.get(j);
    System.out.println(k);
}
```

Accesso elemento per elemento

In alcuni casi un modo di iterare sugli elementi può risultare più utile degli altri. In particolare quando dobbiamo rimuovere un elemento dalla collezione su cui stiamo iterando quello che si prevede di fare è usare un `Iterator`. Infatti **NON** è possibile utilizzare metodi di modifica durante l'iterazione:

```
for (int k : l)
    if (k == x) { l.remove(x); }
```

```
Exception in thread "main" java.util.ConcurrentModificationException
    at java.util.ArrayList$Itr.checkForComodification(Unknown Source)
    at java.util.ArrayList$Itr.next(Unknown Source)
    at ErroriComuni.main(ErroriComuni.java:27)
```

Ma è possibile utilizzare `Iterator.remove()`:

```

Iterator<Integer> i = l.iterator();
while (i.hasNext())
    if (i.next() == x) i.remove();

```

Inoltre, tramite il metodo **forEach** della classe `Iterable` specifica, è possibile iterare su una qualsiasi collezione specificando cosa bisogna fare con i singoli elementi. Infatti `forEach` prende in input un `Consumer`, cioè un'interfaccia funzionale con il solo metodo `void accept(T t)`, che specifica cosa dobbiamo fare di tutti gli elementi.

```

List<Integer> l = List.of(4, 8, 15, 16, 23,
42); l.forEach(x -> System.out.println(x));

```

List<T>

I tipi di `List` principali sono 2:

- **ArrayList**, che ha sostanzialmente lo stesso funzionamento di un array ma a lunghezza dinamica
- **LinkedList**, le classiche liste concatenate a cui si può accedere solo in maniera sequenziale seguendo i vari "puntatori"

Per iterare sulle liste esiste anche un altro tipo di iteratore, chiamato `ListIterator` che estende `Iterator`, e ci permette di scorrere le liste da sinistra a destra:

```

ListIterator<Integer> i = l.listIterator();
while (i.hasNext())
    System.out.println(i.next());

```

O anche da destra a sinistra, specificando un indice da cui partire (per iterare TUTTA la lista basta fornire l'indice all'ultimo elemento):

```

ListIterator<Integer> i = l.listIterator(l.size());
while (i.hasPrevious())
    System.out.println(i.previous());

```

Set<T>

I classici insiemi senza ripetizioni presenti in quasi tutti i linguaggi di programmazione. I principali tipi di set sono:

- **HashSet**: è il tipo di `Set` più comunemente utilizzato in Java, spesso considerato il "set normale". Esso non ha ripetizioni né ordine ed utilizza una tabella hash per memorizzare gli elementi, il che significa che l'elemento viene trasformato in un indice unico tramite una funzione hash per impedire che vi siano ripetizioni.
- **LinkedHashSet**: memorizza gli elementi in ordine di inserimento.
- **TreeSet**: memorizza gli elementi all'interno di un **albero binario di ricerca rosso-nero**. Questo significa che gli elementi vengono memorizzati in un ordine specifico, che può essere l'ordine naturale degli elementi (ad esempio, l'ordine numerico per numeri interi) o un ordine definito da un comparatore.

```

// Creiamo un Comparator per ordinare le stringhe in ordine alfabetico inverso
Comparator<String> reverseOrderComparator = new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return s2.compareTo(s1); // Ordine decrescente
    }
};

// Creiamo il TreeSet passando il comparatore
TreeSet<String> treeSet = new TreeSet<>(reverseOrderComparator);

```

Map<K, V>

Una mappa mette in corrispondenza chiavi e valori (come i dizionari in python). Non può contenere chiavi duplicate, ma uno stesso valore può corrispondere a più chiavi. I principali tipi di map sono:

- **HashMap**: diciamo che è la mappa “normale” che semplicemente memorizza le coppie in una tabella di hash. Non garantisce alcun ordine specifico degli elementi. L'ordine degli elementi può cambiare nel tempo a causa della riorganizzazione interna della tabella hash.
- **LinkedHashMap**: come le HashMap ma mantiene un ordine di inserimento degli elementi.
- **TreeMap**: utilizza un albero rosso-nero per memorizzare le coppie chiave-valore in un ordine naturale o in base a un comparatore fornito.

```
// Creiamo un Comparator per ordinare le chiavi in ordine decrescente
Comparator<Integer> reverseOrderComparator = new Comparator<Integer>() {
    @Override
    public int compare(Integer key1, Integer key2) {
        return key2.compareTo(key1); // Ordine decrescente
    }
};

// Creiamo la TreeMap passando il comparatore
TreeMap<Integer, String> treeMap = new TreeMap<>(reverseOrderComparator);
```

Da una mappa è possibile ottenere:

1. **L'insieme delle chiavi** di una mappa mediante il metodo **keySet()**
2. **L'elenco dei valori**, con ripetizioni, mediante il metodo **values()**
3. **L'insieme delle coppie (chiave, valore)** mediante il metodo **entrySet()** di cui è possibile conoscere quale è la chiave e quale il valore con i metodi **getKey()** o **getValue()**.

Comparable<T>

L'interfaccia **Comparable<T>** è utilizzata per definire un ordinamento naturale degli oggetti di una classe. Quando una classe implementa Comparable<T>, essa deve fornire una definizione per il metodo **compareTo(T o)** che confronta l'oggetto corrente con l'oggetto specificato e restituisce un valore intero che indica l'ordinamento. *(generalmente -1 se this < o, 0 se this = o, 1 se this > o).*

```
public interface Comparable<T> {  
    int compareTo(T o);  
}
```

Implementato direttamente nella classe di un oggetto. Utilizza Comparable quando vuoi definire un ordinamento naturale per una classe, cioè un criterio di ordinamento predefinito che ha senso nella maggior parte dei casi per gli oggetti di quella classe. Questo è particolarmente utile quando l'ordinamento naturale è un aspetto fondamentale della classe. Ad esempio, le classi String, Integer, e Double implementano Comparable<T> per fornire un ordinamento naturale per le stringhe, gli interi e i numeri decimali.

Esempio:

```
public class NomeCognome implements Comparable<NomeCognome>  
{  
    private String nome;  
    private String cognome;  
  
    public NomeCognome(String nome, String cognome)  
    {  
        this.nome = nome;  
        this.cognome = cognome;  
    }  
  
    @Override  
    public int compareTo(NomeCognome o)  
    {  
        int v = cognome.compareTo(o.cognome);  
  
        if (v == 0) return nome.compareTo(o.nome);  
        else return v;  
    }  
  
    @Override  
    public String toString() { return cognome+" "+nome; }  
}
```

Facciamo attenzione però a confrontare sempre tutto quello di cui abbiamo bisogno. Potrebbe sembrare scontato ma non lo è sempre. Nell'esempio qui sopra si nota come sia fondamentale confrontare entrambi i campi prima di stabilire se i due oggetti sono effettivamente uguali.

Comparator<T>

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
  
    boolean equals(Object obj);  
}
```

Usato esternamente alla classe da ordinare, per definire criteri di ordinamento personalizzati. Utilizza Comparator quando non puoi modificare la classe (ad esempio, se la classe fa parte di una libreria di terze parti) o quando desideri ordinare gli oggetti in modi diversi dal loro ordinamento naturale.

Chiaramente, come visto prima per iterator, essendo comparator un'interfaccia non può essere istanziata direttamente. Dunque quello che bisogna fare è creare una classe che implementa comparator ed istanziare quella. Lo possiamo fare in diversi modi:

Classe Esterna o Interna

```
public static class PersonaComparator implements Comparator<Persona> {  
    @Override  
    public int compare(Persona p1, Persona p2) {  
        return p1.getNome().compareTo(p2.getNome());  
    }  
}
```

Classe Anonima

```
Collections.sort(persone, new Comparator<Persona>() {  
    @Override  
    public int compare(Persona p1, Persona p2) {  
        return p1.getNome().compareTo(p2.getNome());  
    }  
});
```

Lambda Expression

```
Collections.sort(persone, (p1, p2) -> p1.getNome().compareTo(p2.getNome()));
```

Metodi di default di Comparator<T>

reversed(): Restituisce un comparatore che impone l'ordine inverso di quello di partenza.

comparing(): crea un comparatore basato su una chiave specifica.

thenComparing(): aggiunge un criterio di ordinamento secondario.

Comparable<T> o Comparator<T>

Solitamente quello che si preferisce fare è utilizzare l'interface Comparable<T> e modificare il metodo compareTo(T o) in base alle esigenze. Tuttavia non è detto che la classe che implementa Comparable sia a noi accessibile e modificabile. Per esempio nel caso delle Collections di Java queste sono librerie fatte e finite. Dunque se vogliamo modificare il modo in cui vengono ordinati dati all'interno di queste collezioni dobbiamo utilizzare un comparator.

Esempio: Ordinare uno scaffale di libri

Ordinamento di default implementato con Comparable<Libro>:

```
public class Libro implements Comparable<Libro>{

    private String titolo;
    private String autore;
    private int pagine;

    public Libro(String titolo, String autore, int pagine) {
        this.titolo = titolo;
        this.autore = autore;
        this.pagine = pagine;
    }

    @Override
    public int compareTo(Libro o) {
        return this.pagine - o.getPagine();

        //oppure usare pagine.compareTo(o.getPagine())
        //MA bisogna usare la classe Wrapper perche compareTo
        //non si puo applicare sui primitivi

        //return Integer.compareTo(pagine, o.getPagine());
    }

    //ordinamento default con l'implementazione di Comparable<Libro>
    Collections.sort(scaffale);
    System.out.println("Ordinato per PAGINE: " + scaffale);
}
```

Ordinamento implementato con CLASSE ESTERNA che implementa Comparator<Libro>

```
public class OrdinaPerTitolo implements Comparator<Libro>{

    @Override
    public int compare(Libro o1, Libro o2) {
        return (o1.getTitolo()).compareTo(o2.getTitolo());
    }
}

//ordinato con il comparatore definito in una CLASSE ESTERNA
scaffale.sort(new OrdinaPerTitolo());
System.out.println("Ordinato per TITOLO: " + scaffale);
```

(con classe interna sarebbe stata la stessa cosa)

Ordinamento con CLASSE ANONIMA Comparator<Libro>

```
//comparatore "al volo" con CLASSE ANONIMA
Comparator<Libro> ordinaPerAutore = new Comparator<Libro>() {
    @Override
    public int compare(Libro o1, Libro o2) {
        return (o1.getAutore()).compareTo(o2.getAutore());
    }
};
scaffale.sort(ordinaPerAutore);
System.out.println("Ordinato per AUTORE: " + scaffale);
```

Ordinamento con Lambda Expression

```
//comparatore al volo con lambda expression
scaffale.sort((l1, l2) -> l1.getAutore().compareTo(l2.getAutore()));
System.out.println("Ordinato per AUTORE: " + scaffale);
```

Esempio: Ordinare uno scaffale di libri con PIU CRITERI

Ordinamento di default implementato con Comparable<Libro>:

Prima in base al titolo. A parità di titolo ordiniamo in base al numero di pagine ed infine in base all'autore.

```
public class Libro implements Comparable<Libro>{

    private String titolo;
    private String autore;
    private int pagine;

    public Libro(String titolo, String autore, int pagine) {
        this.titolo = titolo;
        this.autore = autore;
        this.pagine = pagine;
    }

    @Override
    public int compareTo(Libro o) {
        int result = this.titolo.compareTo(o.getTitolo());

        if (result == 0) {
            result = Integer.compare(this.pagine, o.getPagine());

            if (result == 0) {
                result = this.autore.compareTo(o.getAutore());
            }
        }
        return result;
    }
}
```

Ordinamento tramite Comparator<Libro>:

Per rendere il codice più semplice e leggibile possiamo usare i metodi statici dell'interfaccia Comparator all'interno di un metodo che restituisce il comparator desiderato:

```
public class Libro implements Comparable<Libro>{

    private String titolo;
    private String autore;
    private int pagine;

    public Libro(String titolo, String autore, int pagine) {
        this.titolo = titolo;
        this.autore = autore;
        this.pagine = pagine;
    }

    public static Comparator<Libro> getComparator() {
        return Comparator.comparing(Libro::getTitolo)
            .thenComparing(Libro::getPagine)
            .thenComparing(Libro::getAutore);
    }
}
```

Tipi Generici

La **programmazione generica** in Java è un meccanismo molto potente che permette di scrivere classi, interfacce e metodi che operano su tipi specificati al momento dell'uso, rendendo il codice più riutilizzabile e sicuro in termini di tipi. In pratica, invece di dover creare versioni separate del codice per ogni tipo di dato, puoi scrivere una versione generica che funziona con diversi tipi di dati.

Solitamente il tipo generico viene indicato con una lettera come T, E, K, V, ecc. e su di esso possono essere messi dei “vincoli” in termini di ereditarietà. Possiamo per esempio avere **<T extends Number>** o **<T super Integer>**.

Wildcard “?”

In Java, il jolly “?” viene utilizzato nei tipi generici per rappresentare un tipo sconosciuto o "qualunque tipo". È utilizzato praticamente solo in metodi che prendono in input collezioni, così da rendere quel metodo riutilizzabile qualsiasi sia il tipo.

Type Erasure

Type erasure è il processo utilizzato da Java per rimuovere le informazioni sui tipi generici durante la compilazione. Questo permette di mantenere la compatibilità con le versioni precedenti di Java, che non supportavano i generics. Con type erasure, i parametri di tipo generico vengono convertiti in Object o in un tipo specificato tramite il vincolo extends. Infatti quando dichiariamo una classe generica NON si può utilizzare il vincolo **<T super Number>**.

Eccezioni

Le eccezioni rappresentano un meccanismo utile a notificare e gestire gli errori rendendo il Codice più robusto e sicuro. Le eccezioni “notevoli” e più comuni sono le seguenti:

Eccezione	Descrizione
IndexOutOfBoundsException	Accesso ad una posizione non valida di un array o una stringa (<0 o maggiore della sua dimensione)
ClassCastException	Cast illecito di un oggetto ad una sottoclasse a cui non appartiene. Esempio: <pre>Object x = new Integer(0); System.out.println((String)x);</pre>
ArithmeticException	Condizione aritmetica non valida (es. divisione per zero)
CloneNotSupportedException	Metodo clone() non implementato o errore durante la copia dell'oggetto
ParseException	Errore inaspettato durante il parsing
IOException e IOException	Grave errore di input o output
IllegalArgumentException	Parametro illegale come input di un metodo
NumberFormatException	Errore nel formato di un numero (estende la precedente)

L'onere di gestire i vari tipi di errore si sposta sulla **JVM** che si incarica di capire il modo più opportuno per gestire la situazione di errore.

Che cosa si può gestire con le Eccezioni?

Errori sincroni, che si verificano a seguito dell'esecuzione di un'istruzione

Errori non critici: che derivano da condizioni anomale (es. *divisione per zero, errore di I/O, errori durante il parsing etc.*)

Errori critici o irrecuperabili: interni alla JVM (es. *conversione di tipo non consentito, accesso ad una variabile riferimento con valore null, mancanza di memoria libera, riferimento a una classe inesistente etc...*)

Che cosa NON si può gestire con le Eccezioni?

Eventi asincroni, che accadono parallelamente all'esecuzione e quindi indipendenti dal flusso di controllo (es. *completamenti nel trasferimento I/O, ricezione messaggi su rete, click del mouse etc...*)

Blocco try-catch

Per catturare le eccezioni ed agire di conseguenza si utilizza il **blocco try-catch**.

- Nel **blocco try** si inseriscono tutte le istruzioni che vogliamo “provare” dalle quali vengono sollevate le eccezioni che vogliamo “catturare”.
- Nel **blocco catch** bisogna indicare il tipo di eccezione da catturare e specificare le azioni da attuare a seguito dell’eccezione sollevata.

È chiaramente possibile avere più blocchi catch a seguito di un unico blocco try, così da catturare più eccezioni. Tuttavia è importante sapere che **l’ordine conta!** È fondamentale scrivere i diversi blocchi catch e catturare le eccezioni dalla più specifica a quella più generale.

Nell’attuare il processo di cattura, la JVM sceglie il primo catch compatibile, tale cioè che il tipo dell’eccezione dichiarata sia lo stesso o un supertipo dell’eccezione lanciata durante l’esecuzione. Esempio:

```
public class TavoloDiGioco
{
    public enum Seme
    {
        SPADE,
        DENARI,
        BASTONI,
        COPPE;
    }
    public static void main(String[] args)
    {
        try
        {
            Integer due = Integer.parseInt("Due");
            Seme denari = Seme.valueOf("DENARA");

            Carta dueDiDenari = new Carta(due, denari);
        }
        catch(IllegalArgumentException e)
        {
            System.out.println("Seme non esistente...");
        }
        catch(NumberFormatException e)
        {
            System.out.println("Valore non esistente...");
        }
    }
}
```

- `NumberFormatException` estende `IllegalArgumentException`
- La prima istruzione nel blocco try solleva l’eccezione **più specifica**, la seconda quella **più generale**
- Quando viene eseguita la prima istruzione, viene sollevata una `NumberFormatException`
- Ma nell’analisi la JVM considera il **primo blocco catch** come **compatibile**
- Il secondo caso non viene invece mai raggiunto

Blocco finally

È uno speciale blocco posto dopo tutti i blocchi try-catch che viene **sempre eseguito**, a prescindere dal sollevamento di eccezioni e perfino se nel blocco try-catch vi è un return, un break o un continue.

Tipicamente all’interno del blocco finally vengono eseguite operazioni di clean-up (es. chiusura di eventuali file aperti o rilascio di risorse) in modo da garantire un certo stato dell’esecuzione.

```
public class FileAperto
{
    public static void main(String[] args)
    {
        FileReader fileReader = null;

        try
        {
            fileReader = new FileReader(new File("my/favourite/path"));
            fileReader.read();
        }
        catch (FileNotFoundException e){ e.printStackTrace(); }
        catch (IOException e){ e.printStackTrace(); }
        finally
        {
            try
            {
                fileReader.close();
            }
            catch (IOException e){ e.printStackTrace(); }
        }
    }
}
```



Clean-up!

printStackTrace() e getMessage()

Quando un'eccezione non viene mai catturata viene stampato sul terminale un "riassunto" associato a quell'eccezione, che viene chiamato **stack trace** e contiene una serie di informazioni utili riguardo:

- Il thread in cui l'eccezione è stata sollevata
- Il nome dell'eccezione sollevata
- La successione, in ordine inverso di invocazione, dei metodi coinvolti
- Il file sorgente e il numero di riga di ciascuna invocazione

Questo stack trace viene stampato su console tramite il metodo **printStackTrace()**, offerto dalla classe Throwable. Un altro metodo messo a disposizione dalla stessa classe è **getMessage()**, in grado di restituire, se prevista o disponibile, una descrizione sintetica della ragione per la quale si è verificata l'eccezione. (es. *String Index out of Range*)

Creare Eccezioni Personalizzate

In Java, è possibile creare eccezioni personalizzate estendendo la classe base Exception o RuntimeException (o loro sottoclassi se vogliamo gestire un caso più specifico).

```
public class MyException extends Exception{

    public MyException() {
        super("Inserisci qui il messaggio da stampare");
    }

    //eredita in automatico i metodi stackTrace() e getMessage()
}
```

Eccezioni Checked o Unchecked

Le Eccezioni Checked sono quelle che devono essere dichiarate (con throws) o gestite (con try-catch) durante la compilazione. Se non vengono gestite, il programma non compila. Sono utilizzate quando l'errore potrebbe essere prevedibile e recuperabile.

Le Eccezioni Unchecked NON richiedono di essere dichiarate o gestite dal compilatore. Sono solitamente il risultato di errori di programmazione, che potrebbero essere evitati correggendo il codice. In questa categoria rientrano gli oggetti Error e RuntimeException.

Cosa avviene quando un'eccezione viene lanciata ma non è gestita?

1. **L'eccezione non viene catturata:** Se non esiste un blocco catch corrispondente per l'eccezione lanciata, essa viene considerata come "non gestita".
2. **Propagazione dell'eccezione:** L'eccezione inizia a "risalire" lungo la catena dei metodi. Se l'eccezione viene lanciata all'interno di un metodo, essa viene propagata al metodo chiamante.
3. **Interruzione del programma:** Se l'eccezione non viene gestita nemmeno a livello superiore (ad esempio, nel metodo main()), il programma termina con un errore.
4. **Stampa dello stack trace:** Java stampa lo stack trace dell'eccezione, che fornisce informazioni utili per il debugging, come il tipo dell'eccezione, il messaggio associato, e una lista dei metodi chiamati fino al punto in cui l'eccezione è stata lanciata.

File

L'interazione con i file in Java può avvenire in modalità **buffered** (con buffer) o **unbuffered** (senza buffer), a seconda del tipo di flusso (stream) che utilizzi. La differenza principale tra questi due metodi riguarda il modo in cui i dati vengono letti o scritti, e l'efficienza delle operazioni di input/output (I/O).

Un-Buffered

In modalità unbuffered, i dati vengono letti o scritti direttamente al file ogni volta che viene chiamato un'operazione di lettura o scrittura. Per fare ciò vengono utilizzate le Classi Java: `FileInputStream`, `FileOutputStream`, `FileReader`, `FileWriter`.

- Ogni volta che **read()** viene chiamato, un singolo byte viene letto dal file.
- Ogni volta che **write()** viene chiamato, un singolo byte viene scritto sul file.
- Inefficiente per grandi quantità di dati, poiché ogni operazione richiede accesso diretto al disco.

Buffered

In modalità buffered, i dati vengono letti o scritti in un buffer temporaneo (un array in memoria) piuttosto che direttamente sul file. Il buffer viene svuotato (flushed) solo quando è pieno o quando viene chiuso il flusso (stream). Per fare ciò vengono utilizzate le Classi Java: `BufferedInputStream`, `BufferedOutputStream`, `BufferedReader`, `BufferedWriter`.

- **BufferedInputStream()** legge più byte alla volta in un buffer (tipicamente 8 KB).
- **BufferedOutputStream()** scrive più byte alla volta sul disco.
- Il **flush()** svuota il buffer scrivendo tutti i dati residui che non sono stati ancora inviati.
- Le operazioni di lettura e scrittura sono più veloci perché vengono ridotti i singoli accessi al disco.

Esempio:

```
try (BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream("output.txt")))
{
    bos.write("Hello World!".getBytes());
    bos.flush(); // Assicura che i dati siano scritti immediatamente
} catch (IOException e) {
    e.printStackTrace();
}
```

Scanner

Un'alternativa (più lenta) per leggere dati in un file di testo è usare la classe `java.util.Scanner`.

```
File f = new File("mio_file.txt");
try
{
    Scanner in = new Scanner(f);
    // finche' esiste una prossima riga
    while(in.hasNext())
    {
        // stampa la riga
        System.out.println(in.nextLine());
    }
    // chiude lo scanner e il file
    in.close();
}
catch(FileNotFoundException e)
{
    e.printStackTrace();
}
```

The diagram includes five yellow callout boxes with arrows pointing to specific lines of code:

- Costruisce l'oggetto File** points to `File f = new File("mio_file.txt");`
- Costruisce lo scanner con il file** points to `Scanner in = new Scanner(f);`
- Itera sulle righe di testo del file** points to the `while(in.hasNext())` loop.
- Prossima riga di testo** points to `in.nextLine()` inside the loop.
- Chiude scanner e file** points to `in.close();`

PrintWriter

Un'alternativa per la **scrittura dei dati** è la classe `PrintWriter`. Fornisce versioni sovraccaricate di `print()` e `println()` per scrivere vari tipi di dati come stringhe, numeri interi, numeri a virgola mobile, caratteri e booleani.

```
PrintWriter pw = new PrintWriter(new File("output.txt"));
```

`PrintWriter` può essere utilizzato con un buffer interno per migliorare le prestazioni.

```
PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter("output.txt")));
```

Inoltre sarà possibile attivare il **flush automatico** (svuotamento del buffer) ogni volta che viene invocato un metodo `println()`, `printf()`, o `format()`.

```
PrintWriter pw = new PrintWriter(new FileWriter("output.txt"), true);
```

Esempio di Utilizzo:

```
try (PrintWriter pw = new PrintWriter(new FileWriter("output.txt"))) {
    pw.println("Hello, World!"); // Scrive una stringa e va a capo
    pw.println(42);              // Scrive un intero
    pw.print(3.14);              // Scrive un numero a virgola mobile senza andare a capo
    pw.print(" - Pi greco");
} catch (IOException e) {
    e.printStackTrace();
}
```

Salvare e Caricare dati

In Java il modo più semplice per salvare e caricare i dati è certamente quello che utilizza la “Serializzazione”.

La **serializzazione** in Java è il processo mediante il quale un oggetto viene convertito in un flusso di byte, che può essere successivamente salvato su un file, inviato attraverso la rete o conservato in una base di dati.

La **deserializzazione** è l'operazione inversa, cioè ricostruire l'oggetto originale a partire dal flusso di byte.

Per rendere un Oggetto serializzabile è fondamentale che esso implementi l'**interfaccia Serializable** che non possiede metodi ma funge solo da “segnaposto” a runtime per specificare che quel tipo è serializzabile. Inoltre, ogni oggetto serializzato porta con sé anche tutti i suoi riferimenti. Dunque anche questi dovranno essere serializzabili d implementare l'interfaccia Serializable.

Nel caso in cui alcuni tipi non si vogliano/non si possano salvare (come nel caso degli oggetti Scanner) essi devono essere segnati come **transient**.

L'estensione più comune per i file serializzati java è .ser, ma è possibile utilizzare anche .dat che è più generica per file binari.

Serializzazione – Scrivere i Dati

Questo processo viene reso semplice e possibile tramite due classi principali:

- **FileOutputStream**: Questa classe viene utilizzata per aprire un file e prepararlo per la scrittura dei dati in formato binario. In pratica, crea un collegamento al file su disco.
- **ObjectOutputStream**: Questa classe riceve il FileOutputStream e permette di scrivere i dati che vogliamo su di esso. È la classe specifica per serializzare gli oggetti.
- **writeObject(Object o)**: serializza e scrive l'intero oggetto specificato nel file.

```
public static void salvaGiocatore(Giocatore giocatore, String filePath) {  
    try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(filePath))) {  
        oos.writeObject(giocatore);  
        System.out.println("Salvataggio completato!");  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

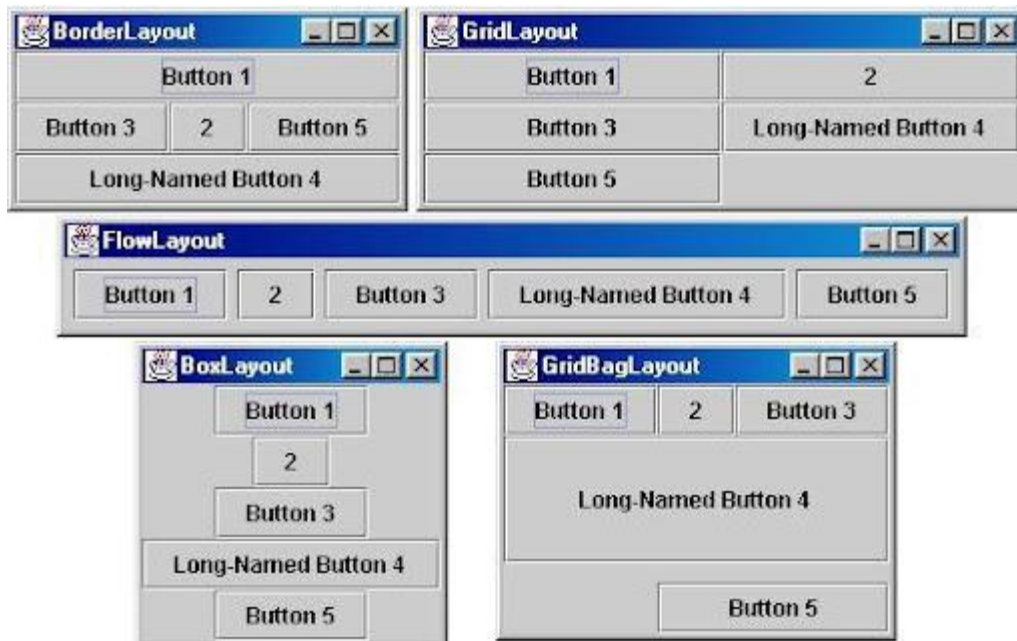
De-Serializzazione – Leggere i Dati

Se per la scrittura usavamo gli outputStream ed il metodo writeObject, per la lettura utilizzeremo:

- **FileInputStream**: apre il file che contiene i dati binari e prepara un flusso di input che prepara i dati dal file per essere letti.
- **ObjectInputStream**: è una classe che può leggere oggetti precedentemente scritti usando ObjectOutputStream. Serve a de-serializzare i byte letti dal file in veri e propri oggetti Java.
- **readObject()**: legge l'oggetto serializzato dal flusso di input e lo restituisce nel rispettivo oggetto Java.

```
public static Giocatore caricaGiocatore(String filePath) {  
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(filePath))) {  
        Giocatore giocatore = (Giocatore) ois.readObject();  
        System.out.println("Caricamento completato!");  
        return giocatore;  
    } catch (IOException | ClassNotFoundException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

Java Swing



Pattern Comportamentali

Pattern Iterator

Fare riferimento a quanto visto riguardo `Iterable<T>` e `Iterator<T>`

Observer Pattern

Model View Controller - MVC

Strategy Pattern

Lo **Strategy Pattern** è un **design pattern comportamentale** che permette di definire una famiglia di algoritmi, incapsularli in classi separate, e renderli intercambiabili. In pratica, ti permette di scegliere e cambiare il comportamento di un oggetto a runtime, senza modificare la logica interna della classe che lo usa.

Implementazione

Step 1: creare un'interfaccia o una classe astratta che definisce il comportamento comune (ovvero i metodi) che ogni strategia concreta dovrà implementare.

```
public interface Strategy {  
  
    int toInt(String s);  
  
}
```

Step 2: Creare le strategie concrete, che estendono o implementano Strategy. Ognuna di queste classi conterrà la propria "strategia", cioè il proprio modo di implementare quel metodo.

```
public class ValoreStrategy implements Strategy{  
  
    @Override  
    public int toInt(String s) {  
        return s.chars().sum();  
    }  
  
}
```

```
public class LunghezzaStrategy implements Strategy{  
  
    @Override  
    public int toInt(String s) {  
        return s.length();  
    }  
  
}
```

```
public class ParseIntStrategy implements Strategy{  
  
    @Override  
    public int toInt(String s) {  
        return Integer.parseInt(s);  
    }  
  
}
```

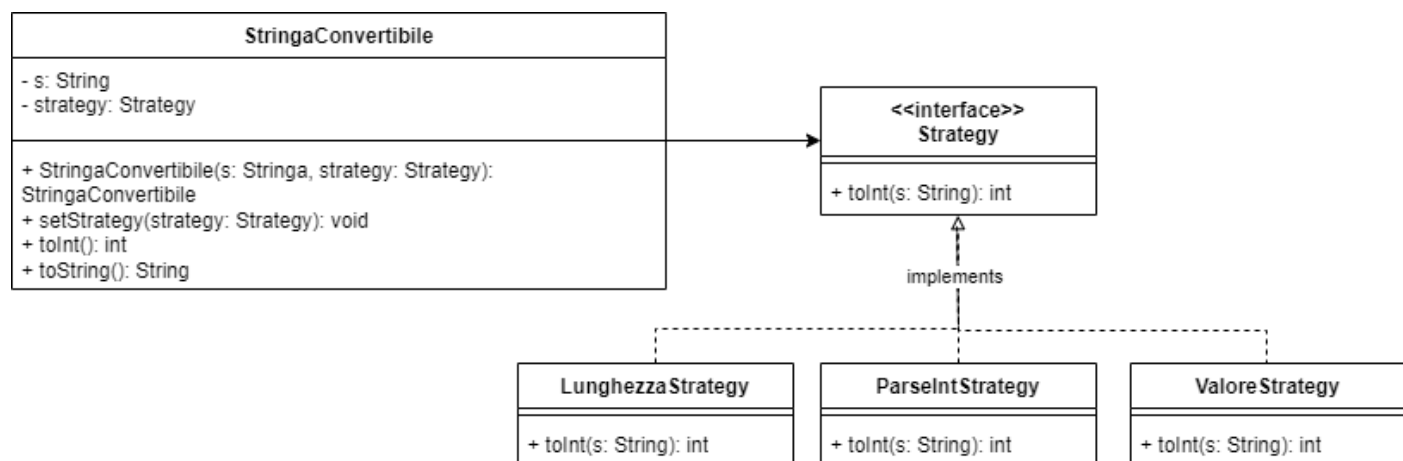
Step 3: creare la classe concreta MyClass (anche detta Context, ovvero quella che dovrà adottare queste strategie) ed inserire al suo interno un riferimento al tipo *Strategia* definito al punto 1. Poi, tramite costruttore o metodi setter, sarà possibile stabilire quale strategia adottare e dunque richiamare i metodi della strategia per definire i comportamenti della classe concreta.

```
public class StringaConvertibile {  
  
    private String s;  
    private Strategy strategy;  
  
    public StringaConvertibile(String s, Strategy strategy) {  
        this.s = s;  
        this.strategy = strategy;  
    }  
  
    public void setStrategy(Strategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public int toInt() {  
        return strategy.toInt(s);  
    }  
  
}
```

Step 4: Utilizzo

```
public static void main(String[] args) {  
  
    StringaConvertibile s1 = new StringaConvertibile("Ciao");  
  
    s1.setStrategy(new LunghezzaStrategy());  
    System.out.println("La lunghezza della stringa è: " + s1.toInt());  
  
    s1.setStrategy(new ValoreStrategy());  
    System.out.println("La somma dei char della stringa è: " + s1.toInt());  
  
}
```

Diagramma Classi



Pattern Creazionali

Singleton Pattern

Il design pattern **Singleton** è uno dei più semplici e comuni design pattern creazionali in programmazione. Lo scopo principale del pattern Singleton è assicurarsi che una classe abbia una sola istanza (oggetto) e fornire un punto globale di accesso a questa istanza. Il pattern Singleton è utile in situazioni in cui:

- Hai bisogno di un solo oggetto per rappresentare una risorsa globale, come ad esempio una connessione al database, un file di log, un gestore di configurazioni, ecc.
- Vuoi controllare l'istanza unica e garantire che non ci siano più copie di quell'oggetto nel sistema.

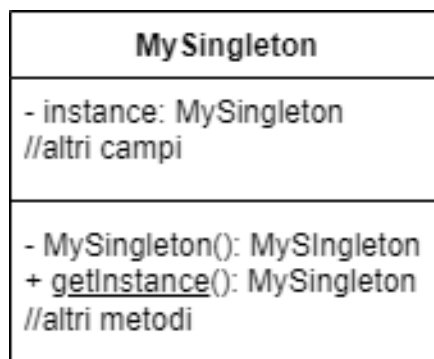
Implementazione

```
public class Singleton {  
    // 1. Istanza privata e statica di se stesso  
    private static Singleton instance;  
  
    // 2. Costruttore privato per evitare la creazione diretta di oggetti  
    private Singleton() {  
        // Codice di inizializzazione  
    }  
  
    // 3. Metodo pubblico e statico che restituisce l'unica istanza  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Tra le caratteristiche principali troviamo:

1. **Costruttore Privato:** Il costruttore della classe Singleton è privato. Questo garantisce che nessuno, al di fuori della classe stessa, possa creare una nuova istanza della classe. In questo modo, l'unica istanza della classe è controllata internamente.
2. **Campo Privato instance:** Questo campo privato memorizza l'istanza unica della classe. È questo oggetto che verrà restituito dal metodo `getInstance()`. Essendo privato, è accessibile solo all'interno della classe Singleton.
3. **Metodo statico `getInstance()`:** Questo metodo pubblico è responsabile di fornire l'accesso all'unica istanza della classe. Controlla se `instance` è null; se lo è, crea una nuova istanza della classe. Se `instance` è già stato inizializzato (non è null), il metodo restituisce semplicemente l'istanza esistente, garantendo così che la stessa istanza sia utilizzata ogni volta che `getInstance()` viene chiamato.

Diagramma Classi



Builder Pattern

Il **Builder Pattern** utilizzato per costruire oggetti complessi in modo step-by-step. È utile quando un oggetto ha molti campi, alcuni opzionali, e si vogliono evitare costruttori troppo complessi o difficili da comprendere. Il Builder pattern separa la costruzione dell'oggetto dalla sua rappresentazione, permettendo di creare varianti dello stesso oggetto con una sintassi più chiara e semplice.

Implementazione

Step 1: Creare la classe principale (Product) che vogliamo costruire. Questa classe dovrà avere un costruttore privato che prende come parametro un oggetto di tipo MyClassBuilder e restituirà un oggetto "clone" del builder.

```
public class Computer {  
    // Proprietà del computer  
    private String processore;  
    private int ram;  
    private int hardDisk;  
    private boolean hasSSD;  
    private String schedaGrafica;  
  
    // Costruttore privato  
    private Computer(ComputerBuilder builder) {  
        this.processore = builder.processore;  
        this.ram = builder.ram;  
        this.hardDisk = builder.hardDisk;  
        this.hasSSD = builder.hasSSD;  
        this.schedaGrafica = builder.schedaGrafica;  
    }  
}
```

Step 2: Creare una classe interna statica MyClassBuilder che contiene un metodo setter per ogni campo della classe principale ed un metodo build che chiama il costruttore privato della classe principale e restituisce l'oggetto desiderato.

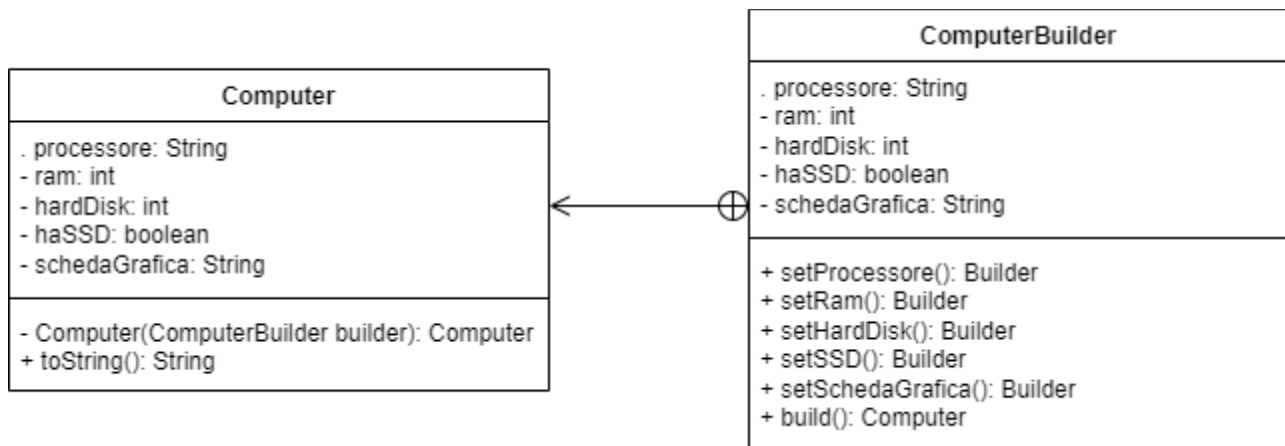
```
public static class ComputerBuilder {  
    // Proprietà del Builder (corrispondenti a quelle del prodotto finale)  
    private String processore;  
    private int ram;  
    private int hardDisk;  
    private boolean hasSSD;  
    private String schedaGrafica;  
  
    // Metodi fluenti per configurare il Computer  
    public ComputerBuilder setProcessore(String processore) {  
        this.processore = processore;  
        return this;  
    }  
  
    public ComputerBuilder setRam(int ram) {  
        this.ram = ram;  
        return this;  
    }  
}
```

```
public Computer build() {  
    return new Computer(this); // Passa il Builder al costruttore privato di Computer  
}
```

Step 3: Utilizzo

```
public static void main(String[] args) {  
    Computer computer = new Computer.Builder()  
        .setProcessore("Intel i7")  
        .setRam(16)  
        .setHardDisk(512)  
        .setHasSSD(true)  
        .setSchedaGrafica("NVIDIA RTX 3080")  
        .build(); // Crea il computer  
}
```

Diagramma Classi



Simple Factory Pattern e Factory Pattern

Il **Factory Pattern** è un pattern di progettazione creazionale che si usa per creare oggetti senza dover specificare la classe esatta dell'oggetto da creare. L'idea di base è delegare a una "factory" la creazione degli oggetti, in modo che il codice client non abbia bisogno di conoscere tutti i dettagli di creazione.

Una **Simple Factory** è solitamente rappresentata da un singolo metodo in una singola classe. Nel tempo, questo metodo potrebbe diventare troppo grande, quindi potresti decidere di estrarre parti del metodo in sottoclassi. Una volta che lo fai più volte, potresti scoprire che il tutto si è trasformato nel classico schema *di metodo* **Factory**.

Implementazione

Step 1: creare un'interfaccia o una classe astratta che verrà implementata o estesa dalle classi concrete. Questa sarà la base comune per tutti gli oggetti che il factory creerà.

```
public abstract class Libro {  
  
    private String titolo;  
    private String genere;  
    private int numPagine;  
    private Scrittore autore;  
  
    public Libro(String titolo, String genere, int numPagine, Scrittore autore) {  
        this.titolo = titolo;  
        this.genere = genere;  
        this.numPagine = numPagine;  
        this.autore = autore;  
    }  
  
    @Override  
    public String toString() {  
        return genere + ": " + titolo + " di " + autore;  
    }  
}
```

Step 2: creare le classi concrete che implementano l'interfaccia o estendono la classe astratta. Queste classi rappresentano i diversi tipi di oggetti che la nostra/e Factory dovrà produrre.

```
public class LibroGiallo extends Libro{  
  
    public LibroGiallo(String titolo, int numPagine, Scrittore autore) {  
        super(titolo, "Giallo", numPagine, autore);  
    }  
}
```

```
public class Fumetto extends Libro{  
  
    public Fumetto(String titolo, int numPagine, Scrittore autore) {  
        super(titolo, "Fumetto", numPagine, autore);  
    }  
}
```

Step 3: creare la classe MyClassFactory che contiene il metodo responsabile della creazione degli oggetti. Questo metodo esaminerà dei parametri o condizioni e deciderà quale oggetto creare.

```
public class LibroFactory {  
  
    public Libro creaLibro(Scrittore scrittore, String titolo, int numPagine) {  
        if (scrittore instanceof ScrittoreGialli) {  
            return new LibroGiallo(titolo, numPagine, scrittore);  
        }  
        else if (scrittore instanceof Fumettista) {  
            return new Fumetto(titolo, numPagine, scrittore);  
        }  
        System.out.println("Genere non supportato");  
        return null;  
    }  
}
```

In questo esempio abbiamo una sola factory (è infatti un SimpleFactory pattern) che in base a delle condizioni sui parametri decide quale oggetto istanziare...

Step 4: Se si vuole implementare il pattern Factory si renda la MyClassFactory astratta o un'interfaccia e si crei una classe separata "OggettoFactory" per ogni oggetto che vogliamo creare.

```
public interface LibroFactory {

    Libro creaLibro(Scrittore scrittore, String titolo, int numPagine);

}

public class GialloFactory implements LibroFactory{

    @Override
    public Libro creaLibro(Scrittore scrittore, String titolo, int numPagine) {
        return new LibroGiallo(titolo, numPagine, scrittore);
    }

}

public class FumettoFactory implements LibroFactory{

    @Override
    public Libro creaLibro(Scrittore scrittore, String titolo, int numPagine) {
        return new Fumetto(titolo, numPagine, scrittore);
    }

}
```

A differenza dell'esempio di prima qui non abbiamo una sola factory contenete la "logica per capire chi istanziare" ma è tutto separato. Ogni classe sa già a quale factory riferirsi per creare i propri oggetti.

Step 5: Utilizzo

```
public static void main(String[] args) {
    Scrittore diGialli = new ScrittoreGialli("Francesco", "Giammaria");
    Libro libro = diGialli.creaLibro("Chi ha mangiato i miei biscotti", 120);
    System.out.println(libro.toString());

    Scrittore fumettista = new Fumettista("Pilar", "Lizzi");
    Libro fumetto = fumettista.creaLibro("Spiderman", 250);
    System.out.println(fumetto.toString());
}
```

Diagramma Classi Simple Factory

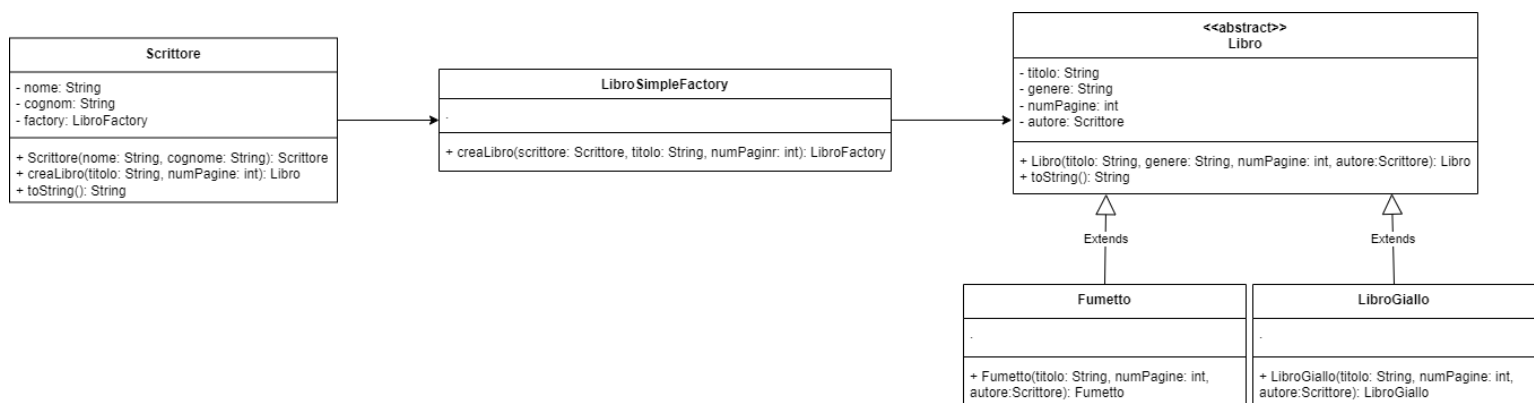
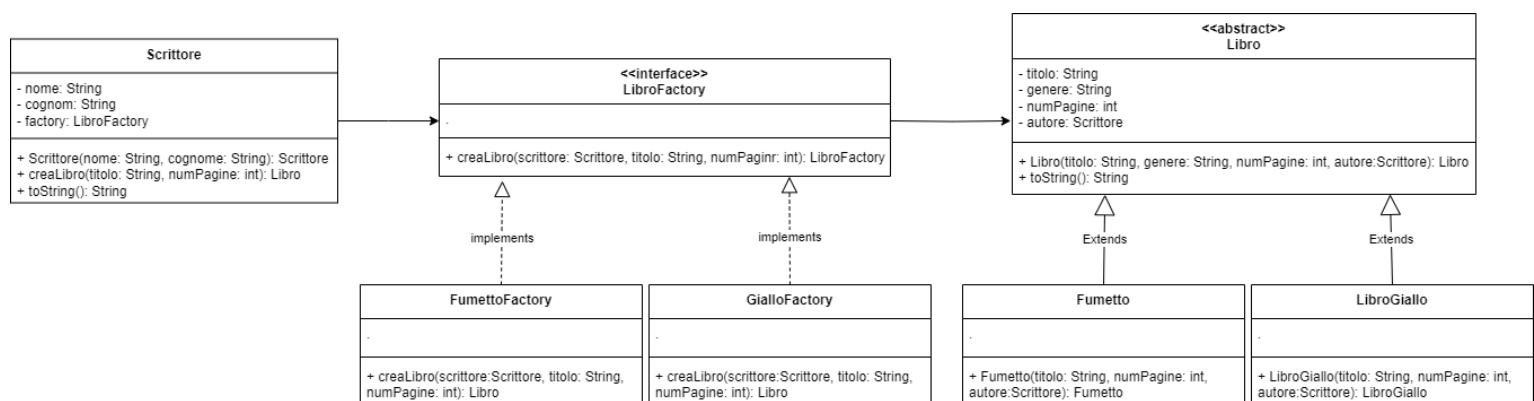


Diagramma Classi Factory



Pattern Strutturali

Decorator Pattern

Il **Decorator Pattern** è un design pattern strutturale che consente di aggiungere dinamicamente nuove funzionalità a un oggetto, senza alterare il suo comportamento originale. Questo pattern è utile quando vuoi estendere le capacità di un oggetto in modo flessibile e scalabile, evitando di creare sottoclassi aggiuntive.

Implementazione

Step 1: definire l'interfaccia comune (Component). Essa conterrà i metodi che vogliamo andare a “decorare”, cioè a modificare ed estendere, man man che decoriamo i nostri oggetti.

```
public interface Notifica {  
  
    void invia();  
  
}
```

Nel caso di una notifica quello che cambierà sarà il modo in cui viene inviata.

```
public interface Caffè {  
  
    String getNome();  
    double getPrezzo();  
  
}
```

Nel caso della caffetteria cambierà il nome della bevande ed il prezzo

Step 2: definire la classe “di base” concreta che vogliamo andare a decorare. Essa deve necessariamente implementare l'interfaccia Component dello step 1...

```
public class CaffèBase implements Caffè{  
  
    private String nome;  
    private double prezzo;  
  
    public CaffèBase() {  
        this.nome = "Caffè";  
        this.prezzo = 0.99;  
    }  
  
    @Override  
    public String getNome() {  
        return nome;  
    }  
  
    @Override  
    public double getPrezzo() {  
        return prezzo;  
    }  
  
}
```

Step 3: creare una classe astratta MyClassDecorator, che implementa l'interfaccia Component e mantiene un riferimento all'oggetto da decorare. Questa classe non fa nulla di speciale. Semplicemente richiama i metodi “già decorati” del nostro oggetto.

```
public abstract class CaffèDecorator implements Caffè{  
  
    protected Caffè caffèDecorato;  
  
    public CaffèDecorator(Caffè caffèDecorato) {  
        this.caffèDecorato = caffèDecorato;  
    }  
  
    @Override  
    public String getNome() {  
        return caffèDecorato.getNome();  
    }  
  
    @Override  
    public double getPrezzo() {  
        return caffèDecorato.getPrezzo();  
    }  
  
}
```

Step 4: creare una classe concreta Decorator. Questa classe estende la MyClassDecorator astratta e semplicemente ne richiama i metodi già decorati e li estende ed aggiungendoci “qualcosa”.

```
public class CaffeMacchiato extends CaffeDecorator{

    public CaffeMacchiato(Caffe caffeDecorato) {
        super(caffeDecorato);
    }

    public String getNome() {
        return super.getNome() + " Macchiato";
    }

    public double getPrezzo() {
        return super.getPrezzo() + 0.10;
    }

}

public class CaffeConTantoLatte extends CaffeDecorator{

    public CaffeConTantoLatte(Caffe caffeDecorato) {
        super(caffeDecorato);
    }

    public String getNome() {
        return super.getNome() + " con Tanto Latte";
    }

    public double getPrezzo() {
        return super.getPrezzo() + 0.50;
    }

}

public class CaffeECornetto extends CaffeDecorator{

    public CaffeECornetto(Caffe caffeDecorato) {
        super(caffeDecorato);
    }

    public String getNome() {
        return super.getNome() + " e Cornetto";
    }

    public double getPrezzo() {
        return super.getPrezzo() + 0.50;
    }

}
```

Step 5: Utilizzo

```
public static void main(String[] args) {
    //caffè base
    Caffè caffè = new CaffèBase();
    caffè.stampa();

    //iniziamo a decorare il nostro caffè
    caffè = new CaffèMacchiato(caffè);
    caffè = new CaffèConTantoLatte(caffè);
    caffè = new CaffèECornetto(caffè);
    caffè.stampa();
}
```

```
//è anche possibile fatto tutto quanto anche con una sola riga:
Caffè caffèConTantoLatteEMacchiato = new CaffèECornetto(new CaffèConTantoLatte(new CaffèMacchiato(new CaffèBase())));
caffèConTantoLatteEMacchiato.stampa();
```

Diagramma Classi

